

第 12 章 | 從零開始實作多層神經網路：MNIST 手寫辨識

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線與學習目標

在自動化深度學習框架（如 TensorFlow 或 PyTorch）普及的今日，許多開發者習慣於直接調用高階 API。然而，從零開始（From Scratch）實作多層神經網路有助於理解前向傳播、權重更新與反向傳播的計算流程。歷史上，神經網路曾因無法有效訓練多層結構而陷入「AI 寒冬」，直到 1986 年 Rumelhart、Hinton 與 Williams 等人重新發現並推廣了反向傳播演算法（Backpropagation），並成為現代深度學習的重要基礎。

本章從 Adaline 延伸到多層神經網路；手動建立權重矩陣並實作鏈式法則，能幫助你理解收斂問題與超參數影響。

學習目標

完成本章後，你將具備以下四大核心能力：

解析數學結構：能精確描述多層感知器（MLP）的層級互動，並推算權重矩陣的維度變化。

掌握反向傳播：理解如何利用鏈式法則進行「責任分配」，將輸出誤差回溯至每一層的權重更新。

原始資料處理：具備讀取 MNIST 原始位元格式（Byte format）資料，並處理大端序（Big-endian）與位元解碼的能力。

模型實務診斷：能透過訓練與驗證曲線判斷過擬合（Overfitting）時機，並應用 L2 正規化等策略進行最佳化。

在深入程式碼前，我們必須先建立對神經網路運作邏輯的核心直覺。

2. 核心觀念白話說明

神經網路的演進源於對生物神經元的模仿。雖然早期的感知器曾遭遇困境，但多層感知器（MLP）的出現讓複雜問題的解決成為可能。

多層感知器的結構

從 Adaline 到 MLP：單層的 Adaline 像是一位「線性決策者」。MLP 則由多個神經元依層次傳遞並轉換訊號。

非線性激活函數的實務意義：為何需要 Sigmoid 函數？若沒有非線性激活，多層網路不論堆疊幾層，其數學本質終究會「塌陷」成一個簡單的線性模型。Sigmoid 的 S 型曲線將輸出映射至 (0, 1) 區間，這在「一對多（OvA）」的多類別預測中可將輸出解讀為機率，能將輸出解讀為機率。

隨機梯度下降（SGD）的雜訊益處：深度學習的成本表面（Cost Surface）充滿了局部最小值。SGD 在處理小批次資料時帶來的「雜訊」，反而能幫助模型實務性地「跳出」局部最小值陷阱，尋找更佳的全局解。

反向傳播的「責任分配」直覺

反向傳播本質上是數學上的鏈式法則，但在學習直覺上，它可以被視為一種誤差責任制：根據最終預測的偏差，從輸出層往回推算，判定每一層的權重應該為該誤差負起多少「責任」，並據此修正權重。

3. 重要術語中英對照表

英文術語	中文譯名	核心定義與實務特性
Multilayer Perceptron (MLP)	多層感知器	由輸入層、一個或多個隱藏層及輸出層組成的完全連接前饋網路。
Forward Propagation	前向傳播	訊號從輸入層穿過隱藏層，最終在輸出層產生預測值的過程。
Backpropagation	反向傳播	利用鏈式法則計算各層權重梯度的核心演算法，旨在最小化誤差。
Bias Unit	偏置單元	數學上常設為 1 的常數項，賦予模型平移激活函數的能力（程式碼中常獨立處理）。
Net Input (z)	淨輸入	輸入特徵與權重的加權總和， $z=WTx+b$ 。
Stochastic Gradient Descent (SGD)	隨機梯度下降	具備高計算效率，且其雜訊特性有助於逃離局部最小值陷阱。
One-hot Encoding	獨熱編碼	將類別標籤（如數字 7）轉換為向量形式（如 $[0,0,0,0,0,0,1,0,0]$ ）。
L2 Regularization	L2 正規化	透過懲罰過大的權重值（L2 範數）來控制模型複雜度，減輕過擬合。

4. 模型架構與數學邏輯

在實作 `NeuralNetMLP` 時，理解權重矩陣的維度（Dimensions）是成功與否的關鍵。

層級結構與維度對照

假設輸入特徵數為 m ，隱藏層單元為 d ，輸出類別數為 t ：

輸入層：激活值為 $a(in)$ 。

隱藏層：權重矩陣 $W(h) \in R^{m \times d}$ 。注意：在程式碼實作中，偏置項 b_h 通常作為獨立的 $1 \times d$ 向量處理，不計入 $W(h)$ 維度。

輸出層：權重矩陣 $W(out) \in R^{d \times t}$ 。

激活函數與溢出處理

我們使用 Sigmoid 函數 $\phi(z)=1+e^{-z}$ 。在底層實作時，為了防止 e^{-z} 產生數值溢出，原始碼中使用了 `np.clip(z, -250, 250)`。這是一個專業的工程細節，確保模型在訓練初期權重劇烈波動時仍能穩定運作。

5. 程式碼流程導讀：NeuralNetMLP 類別

權重初始化方法

在 `fit` 方法中，權重初始化並非隨機即可。

權重 (W)：使用 `normal(loc=0.0, scale=0.1)` 進行高斯分布初始化。這能打破對稱性，避免神經元學習到重複特徵。

偏置 (b)：初始值設為 `zeros`。

核心方法拆解

`_forward`：執行四步運算： $z_h \rightarrow a_h \rightarrow z_{out} \rightarrow a_{out}$ 。

`_compute_cost`：同時計算邏輯損失（Logistic Loss）與 L2 正規化項。關鍵細節：偏置項 b 不參與 L2 處罰。在梯度更新時， $\Delta b_h = \text{grad}_b h$ ，而不像權重項需加上 λw 。

反向傳播實作：

計算輸出層誤差： $\delta(\text{out}) = a(\text{out}) - y$ 。

計算隱藏層誤差： $\delta(h) = (\delta(\text{out})(W(\text{out}))^T \odot (a(h) \odot (1 - a(h))))$ 。

利用這些誤差計算梯度並更新權重。

6. Google Colab 實作與資料預處理

MNIST 原始格式處理

MNIST 資料並非直接可讀的圖像，而是字節流。使用 `struct.unpack` 時需注意：

Magic Number：標籤檔為 2049，圖像檔為 2051。

Endianness：必須指定 `>`（Big-endian，大端序），這是因為資料存儲格式為 MSB first。

預處理公式

為了最佳化梯度下降的穩定性，必須將像素（0-255）進行歸一化：

```
images = ((images / 255.) - .5) * 2 # 縮放至 [-1, 1] 區間
```

建議超參數

`n_hidden = 100`

`epochs = 200`

`eta = 0.0005`

`minibatch_size = 100`

7. 常見錯誤與模型評估

過擬合的教科書案例

參考訓練曲線，當 Epoch 達到約 50 之後，訓練準確率（Training Acc）持續攀升，但驗證準確率（Validation Acc）卻開始持平甚至微幅震盪。這代表模型開始「背誦」訓練集。

診斷與對策

診斷：若兩條曲線差距擴大，即為過擬合。

對策：

提高 `l2` 參數（例如從 0.01 調升至 0.1）。

增加訓練資料或考慮 Dropout 技術。

提早停止（Early Stopping）。

8. 動手練習與自我檢查表

動手做任務

結構變更：將 `n_hidden` 改為 50，觀察其對收斂速度與最終準確率的影響。

錯誤樣本診斷：篩選出預測錯誤的樣本。請找出「看起來像 0 的 6」或「看起來像 9 的 8」。來源文本指出，有些手寫變體（如數字 7 的多種寫法）連人類都難以判定，這能幫助你理解模型的性能極限。

自我檢查表

- ☐ 我是否能解釋為什麼 MNIST 需要展平成 784 維向量？
- ☐ 我是否理解為什麼 `np.clip(z, -250, 250)` 能防止 Sigmoid 溢出？
- ☐ 我是否能說明 Sigmoid 的微分公式 $a(1-a)$ 是如何匯出並應用在反向傳播中的？
- ☐ 我是否理解偏置項（Bias）在程式碼中不參與正規化的原因？

本章總結：理解從零實作的前向傳播與反向傳播後，你能更有依據地診斷 TensorFlow 或 PyTorch 模型的訓練問題。